A Gentle Introduction to Python for Research Computing

Ben Keene

University of Central Florida

22 June 2026

License & Attribution

This material is adapted from **The Carpentries**
(Software Carpentry, Data Carpentry, Library Carpentry).

Licensed under **Creative Commons Attribution 4.0 International**
(CC BY 4.0).

You are free to *share* and *adapt* this material for any purpose, provided you give appropriate credit.

https://carpentries.org/

Overview

In this beginner-friendly session, we will cover the core Python concepts for research workflows, including data handling, basic analysis, and scripting.

Why Python for Research?

- High-level language, easy to learn
- Large ecosystem of scientific libraries
- Strong community support
- Great for automation and reproducibility

However, Python can be slow for certain tasks, better to use C or Fortran for scientific computing.

In ML/Deep Learning, Python libraries like PyTorch and TensorFlow leverage C++/CUDA for performance-critical operations.

Characterizing Research Workflows

- Data generation/collection
- Data processing and analysis
- Visualization and reporting

Data Generation & Collection

Data Generation: Simulations, experiments, synthetic data used for training machine learning models.
Data Collection: Gathering data from various sources, manually downloading datasets, using APIs, or scraping data from the web.

- Tabular data: CSV, Excel, SQL databases
- Image data: JPEG, PNG, TIFF
- Text data: JSON, XML, plain text

Data Processing & Analysis

Data Processing: Cleaning, transforming, and preparing data for analysis.
Data Analysis: Probably your specific research needs.

- Training ML models and evaluating their performance.
- Simulating various combinations of chemical reactions.
- Evaluating the effectiveness of different approaches to a research question.

Visualization & Reporting

Visualization: Creating plots, charts, and other visual representations of data.

- Matplotlib, Seaborn, Plotly for plotting
- Jupyter Notebooks for interactive data analysis

Reporting: Saving the output of your analysis in a format that can be distributed. Key for reproducibility and sharing results.

- Text files for documentation, logging, saving configurations.
- Auto-generate a LaTeX document for publication-quality output.

Getting Started

Navigate to the following URL to get started:
https://bit.ly/rcijupyter.

Your username is the first letter of your first name, and your full last name.

$$\text{John Doe} \mapsto \text{jdoe}$$

$$\text{Ima Knight} \mapsto \text{iknight}$$

Remember your password!

Your password will be set to whatever you enter in the password field. Make sure to remember it for future use.

Our Scenario: A Miracle Arthritis Cure?

Dr. Maverick claims a new drug cures arthritis inflammation flare-ups within **3 weeks** of a patient's first flare-up.

After months of asking, we finally have the clinical trial data: a single CSV spreadsheet.

Our job is to check the claim. To do that, we will:

- Calculate the average inflammation per day across all patients.
- Plot the result to discuss and share with colleagues.

This is a typical research workflow: *collect* $\rightarrow$ *analyze* $\rightarrow$ *visualize* $\rightarrow$ *conclude*.

Interpreting the Data

The trial followed **60 patients** over **40 days**, stored as comma-separated values (CSV):

- Each **row** is one patient.
- Each **column** is one day of the trial.
- Each **value** is the number of inflammation flare-ups that patient had on that day.

```
0,0,1,3,1,2,4,7,8,3,3,3,10,5,7,4,7,...
0,1,2,1,2,1,3,2,2,6,10,11,5,9,4,4,7,...
0,1,1,3,3,2,6,2,5,9,5,7,4,5,4,15,5,...
```

Row 3, column 7 is 6: patient 3 had 6 flare-ups on day 7. Patients start medication after their first flare-up, so we expect inflammation to rise, then fall as the drug takes effect.

Live Demonstration

Within JupyterLab, open a new terminal. Type `python` and press Enter.

Demo Reference

```
# shell
python

# Python REPL
>>> 3 + 5 * 4
23
>>> weight_kg = 60
>>> print(weight_kg)
60
```

Data Types

- `int`: Integer values.
- `float`: Floating-point values.
- `str`: String values.
- And more...

Using Variables

```
>>> x = 5
>>> y = 10
>>> z = x + y
>>> print(z)
15
>>> first_name = 'Ben'
>>> last_name = 'Keene'
>>> full_name = first_name + ' ' + last_name
>>> print(full_name)
Ben Keene
```

Printing Output

- Use the `print()` function to display output.
- You can print strings, numbers, and other data types.

Either: `print(your_variable)` or
`print("Your message:", your_variable)`.

Fancy formatting using f-strings:
`print(f"Your message: {your_variable}")`.

Functions

We've already used the built-in function `print()`.

`type()` returns the type of an object.
`help()` provides help information for a function or module.

You can define your own functions using the `def` keyword.

```
>>> def my_function():
>>>     print("Hello, World!")
>>> my_function()
Hello, World!
```

Loading Data

Make sure you are inside the `beginner-python` directory in your shell.

If you are lost, you can enter `cd ~/beginner-python`.

Loading Data 2

```
>>> import numpy as np
>>> np.loadtxt(fname='data/inflammation-01.csv', delimiter
    =',')
>>> data = np.loadtxt(fname='data/inflammation-01.csv',
    delimiter=',')
>>> print(data)
...
>>> print(type(data))
<class 'numpy.ndarray'>
...
>>> print(data.dtype)
float64
>>> print(data.shape)
(60, 40)          # 60 rows 40 columns
```

Visualizing the Data

Let's use a Jupyter notebook (ipynb) to visualize the data.

From the notebook, we will import Matplotlib and create some plots to investigate the claim that the medicine is effective.

A First Plot: The Heat Map

If you started a fresh notebook, reload the data first:

```
import numpy as np
data = np.loadtxt(fname='data/inflammation-01.csv',
    delimiter=',')
```

Now import Matplotlib and draw a heat map of the whole dataset:

```
import matplotlib.pyplot as plt
image = plt.imshow(data)
plt.show()
```

Each **row** is a patient, each **column** a day. Blue is low, yellow is high.

Reading the Heat Map

The heat map matches Dr. Maverick's story *so far*:

- Patients start medication once flare-ups begin.
- It takes around 3 weeks for the drug to take effect.
- Flare-ups appear to drop to zero by the end of the trial.

But a single picture can hide a lot. Let's look at some summary statistics *across all patients*, one value per day.

Average Inflammation Over Time

`np.mean(data, axis=0)` collapses the 60 rows into a single average for each of the 40 days:

```
ave_inflammation = np.mean(data, axis=0)
ave_plot = plt.plot(ave_inflammation)
plt.show()
```

A reasonably linear rise and fall — consistent with the claim that the medication takes about 3 weeks to work.

A good data scientist never stops at the average.

Maximum and Minimum

```
max_plot = plt.plot(np.max(data, axis=0))
plt.show()

min_plot = plt.plot(np.min(data, axis=0))
plt.show()
```

- The **maximum** rises and falls in clean straight lines.
- The **minimum** looks like a step function.

Neither trend is biologically plausible. Either our calculation is wrong, or *something is wrong with the data*.

Grouping Plots: Subplots

We can place several plots in one figure to compare them at a glance.

```
fig = plt.figure(figsize=(10.0, 3.0))
axes1 = fig.add_subplot(1, 3, 1)
axes2 = fig.add_subplot(1, 3, 2)
axes3 = fig.add_subplot(1, 3, 3)

axes1.set_ylabel('average')
axes1.plot(np.mean(data, axis=0))
axes2.set_ylabel('max')
axes2.plot(np.max(data, axis=0))
axes3.set_ylabel('min')
axes3.plot(np.min(data, axis=0))

fig.tight_layout()
plt.savefig('inflammation.png')
plt.show()
```

Understanding `add_subplot`

`plt.figure(figsize=(w, h))` creates the canvas.

`fig.add_subplot(rows, cols, n)` adds one plot:

- **rows**: total rows of subplots
- **cols**: total columns of subplots
- **n**: which cell, counting left-to-right, top-to-bottom

Optional: Lists

Walk through lists, slicing, and other list operations.

Optional: Loops

Walk through `for` loops and how to use them with lists.

Analyzing Many Files at Once

Dr. Maverick eventually hands over **twelve** CSV files, one per clinical trial. We don't want to copy-paste our analysis twelve times.

Instead, we'll:

- Get a list of every file whose name matches a pattern.
- Write a `for` loop to run the same analysis on each one.

This is the payoff of writing reusable code: the work scales from one file to many for free.

Finding Files with glob

The glob library finds files whose names match a pattern:

- * matches zero or more characters.
- ? matches any single character.

```
import glob
print(glob.glob('data/inflammation*.csv'))
['data/inflammation-05.csv', 'data/inflammation-11.csv',
 'data/inflammation-12.csv', 'data/inflammation-08.csv',
 ...
 'data/inflammation-01.csv']
```

glob.glob returns a **list** in arbitrary order, so we can loop over it.

Sorting the File List

The order is arbitrary, so we use `sorted()` to get a predictable, alphabetical list. We'll start with just the first three files:

```
import glob
import numpy as np
import matplotlib.pyplot as plt

filenames = sorted(glob.glob('data/inflammation*.csv'))
filenames = filenames[0:3]
```

`sorted()` returns a *new* sorted list; slicing `[0:3]` keeps the first three entries.

One Loop, Many Files

```
for filename in filenames:
    print(filename)
    data = np.loadtxt(fname=filename, delimiter=',')

    fig = plt.figure(figsize=(10.0, 3.0))
    axes1 = fig.add_subplot(1, 3, 1)
    axes2 = fig.add_subplot(1, 3, 2)
    axes3 = fig.add_subplot(1, 3, 3)

    axes1.set_ylabel('average')
    axes1.plot(np.mean(data, axis=0))
    axes2.set_ylabel('max')
    axes2.plot(np.max(data, axis=0))
    axes3.set_ylabel('min')
    axes3.plot(np.min(data, axis=0))

    fig.tight_layout()
    plt.show()
```

Comparing the Results

The loop produces one figure per file. Looking across them:

- Files 1 and 2 look almost identical: the same suspiciously clean maxima and minima we saw before.
- File 3 has much *noisier* average and maximum plots — actually *less* suspicious.
- But file 3's minimum is stuck at **zero** for every single day.

A heat map of file 3 shows zeros scattered throughout, and one patient with *no* flare-ups at all — they may not even have arthritis.

The Verdict

Investigating all twelve files, two categories emerge:

- "Ideal" datasets that match the claim perfectly — but have impossible, too-clean maxima and minima.
- "Noisy" datasets with sporadic missing values and unsuitable participants.

Worse, three of the noisy files (03, 08, 11) are **identical down to the last value**.

Confronted, Dr. Maverick admits the data was **fabricated** — fake datasets, padded out with duplicate copies of a bad one.